Console-Based Application in C++

Project Report

March 2026

Course	Programming Fundamentals / C++
Language	C++ (ISO C++11 / C++14)
Paradigm	Procedural Programming
Topic	Functions, Loops, Input Validation

1 Introduction

This project implements a simple console-based banking system in C++. The primary objective is to demonstrate core procedural programming concepts including user-defined functions, control structures, input validation, and formatted output. The application simulates basic banking operations that a real-world ATM or teller terminal would provide.

The program is structured around a menu-driven loop, which allows the user to interactively perform operations until they choose to exit. All business logic is encapsulated in separate functions, making the code modular and easy to maintain.

2 Objectives

- Implement a menu-driven interface using a **do-while** loop.
- Practise function declaration, definition, and calling conventions in C++.
- Apply input validation to prevent invalid or out-of-range operations.
- Use **<iomanip>** to format monetary values to two decimal places.
- Handle edge cases such as overdraft attempts and negative deposit amounts.
- Manage the input buffer reliably to prevent input-stream corruption.

3 System Design

3.1 Program Flow

The program begins by initialising the balance to a default value (**$123.04**) and entering a **do-while** loop. Inside the loop a four-option menu is displayed. The user's choice is read and processed by a **switch** statement that delegates work to one of three helper functions. The loop continues until the user selects option 4 (Exit).

Step	Action	Function Called
1	Display menu & read choice	main()
2	Show current balance	showbalance(balance)
3	Accept deposit	deposit()
4	Process withdrawal	withdraw(balance)
5	Exit program	—

Table 1 — Program execution flow

3.2 Function Summary

Function Signature	Return Type	Responsibility
showbalance(double balance)	void	Prints the formatted balance.
deposit()	double	Reads and validates a deposit amount.
withdraw(double balance)	double	Validates and returns withdrawal amount.

Table 2 — Function signatures and responsibilities

4 Implementation Details

4.1 Main Loop and Menu

A **do-while** loop is used instead of a **while** loop so that the menu is always displayed at least once. The **balance** variable is declared outside the loop to persist across iterations — declaring it inside would reset the balance on every pass.

```
do {
std::cout << "1. Show Balance\n";
std::cout << "2. Deposit\n";
std::cin >> choice;
// clear input buffer
std::cin.clear();
fflush(stdin);
switch (choice) { ... }
} while (choice != 4);
```

4.2 Input Buffer Management

After reading an integer with **std::cin**, the newline character remains in the buffer. If left uncleaned it causes subsequent reads to fail silently. Two calls are used to mitigate this:

```
std::cin.clear(); // resets error-state flags
fflush(stdin); // clears remaining characters in the buffer
```

Note: **fflush(stdin)** is technically undefined behaviour in the C++ standard. A portable alternative is **std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')**.

4.3 Deposit Validation

The **deposit()** function ensures only a positive value is accepted. If the user enters zero or a negative number the function returns 0, leaving the balance unchanged.

```
double deposit() {
double amount = 0;
std::cin >> amount;
if (amount > 0) return amount;
else { std::cout << "Invalid amount"; return 0; }
}
```

4.4 Withdrawal Validation

The **withdraw()** function guards against two failure modes: attempting to withdraw more than the available balance (overdraft), and entering a negative value. Only when neither condition is true does it return the requested amount.

```
double withdraw(double balance) {
double amount = 0;
std::cin >> amount;
if (amount > balance) { std::cout << "Insufficient funds"; return 0; }
else if (amount < 0) { std::cout << "Invalid amount"; return 0; }
else return amount;
}
```

4.5 Formatted Output

Monetary values are always displayed with exactly two decimal places by combining **std::fixed** and **std::setprecision(2)** from the **<iomanip>** header.

```
void showbalance(double balance) {
std::cout << "Balance: $"
<< std::fixed << std::setprecision(2)
<< balance << '\n';
}
```

5 Testing & Validation

The following test cases were executed to verify correctness of each feature.

#	Test Case	Input	Expected Output	Result
1	Show initial balance	Choice: 1	Balance: $123.04	PASS
2	Valid deposit	$50.00	Balance: $173.04	PASS
3	Negative deposit	-$20	Invalid amount, no change	PASS
4	Zero deposit	$0	Invalid amount, no change	PASS
5	Valid withdrawal	$100.00	Balance: $73.04	PASS
6	Overdraft attempt	$500.00	Insufficient funds	PASS
7	Negative withdrawal	-$10	Invalid amount	PASS
8	Invalid menu option	9	Invalid case	PASS
9	Non-numeric menu input	abc	No crash, re-prompts	PASS
10	Exit	4	Thanks for visiting	PASS

Table 3 — Test cases and results

6 Limitations

- **No data persistence** — balance resets to $123.04 each time the program is run.
- **Single account** — the system does not support multiple users or accounts.
- **Hardcoded initial balance** — the starting value is embedded in the source code.
- **fflush(stdin) portability** — this call is undefined behaviour per the C++ standard and may not work on all compilers.
- **No transaction history** — past deposits and withdrawals are not recorded.

7 Future Enhancements

- Save and load balance from a **text file** using file I/O streams.
- Replace **fflush(stdin)** with **std::cin.ignore()** for standard-compliant buffer flushing.
- Add a **PIN authentication** system to simulate real-world security.
- Support **multiple accounts** stored in an array or **std::vector**.
- Implement a **transaction log** to record all deposits and withdrawals with timestamps.

- Refactor into an **object-oriented design** using a BankAccount class.

8 Conclusion

This project successfully demonstrates the application of fundamental C++ programming concepts through a practical, real-world scenario. The banking system implements a clean menu-driven interface, robust input validation, and modular function design — all key competencies in procedural programming.

The program correctly handles all identified edge cases including overdraft attempts, negative inputs, and non-numeric user entries. All ten test cases passed without error. The codebase is well-structured and serves as a solid foundation for future object-oriented enhancements.

End of Report